

Тема 11. ВИЗУАЛИЗАЦИЯ НА ТРИМЕРНИ ОБЕКТИ Математически инструменти и моделиране на гладки форми и повърхности. Алгоритми за визуализация на тримерни обекти. Стандартна система от проективни преобразувания. OpenGL - Проекции и работа с квадратични обекти

упр. 12 Системи с тримерни изображения. Реализация на паралелни и центални проекции на повърхности. OpenGL -3D трансформации и квадратични обекти

11.1. ОСНОВИ НА ПРЕДСТАВЯНЕТО НА ТРИМЕРНИ ГРАФИЧНИ ОБЕКТИ

Синтезът на тримерни изображения в компютърната графика проектиране на тримерни повърхности, премахване на скрити линии и текстура, създаваща впечатление за дълбочина. Основните алгоритми тук са за методите за отделянето на видимите и невидими елементи на сцената, алгоритмите за щриховка, методите за рандомизация, използващи придаването на реалистичен вид на възпроизвежданото изображение. Тук се включват алгоритмите и за представяне на прозрачност, мъгла и сенки.

11.1.1 Задаване на тримерни повърхности

В редица приложения на компютърната графика възниква необходимостта за генериране и използване на графични изображения в тримерната (3D) форма на представяне. Това е нужно по-конкретно в редица компютърни технологии като: проектиране, конструиране, управление на тримерни технологични процеси и др. За решението на подобни задачи е необходимо най-напред да се опишат и създадат 3D повърхности в пространството. Такова представяне на тримерните изображения се нарича скелетно.

При възпроизвеждане на изображението на сложни тримерни обекти е необходимо да се разполага с тяхното математическо описание. Прилагат се два начина. Първият предполага изчертаване на множество криви на реалния тримерен обект и използването на математическото им описание за възпроизвеждане на изображението. Този метод е удобен да се използва при работа на векторни графични устройства и изображението напомня на мрежа от жици. Това е един от най- разпространените подходи за скелетно-тримерно представяне на повърхности в пространството [11], а именно чрез **много-ъгълникова мрежа** (полигонална мрежа). Полигоналната мрежа представлява съвкупност от свързани по между си равнинни многоъгълници. Задаването на такава мрежа се осъществява много лесно чрез нейните върхове и ръбове. Един от основните недостатъци на този подход е неговата приблизителност.

Вторият начин предполага разбиване на повърхността на тялото на някакъв брой **крайни участъци**, ограничени със затворени криви. За всеки от тези участъци може да се намери приближено математическо описание и изображението на обекта се възпроизвежда като съвкупност от изображения на тези крайни участъци от повърхности, ограничени със затворени криви. В качеството на такива крайни участъци често се използват плоски **триъгълници**, тъй като триангулацията на повърхност е достатъчно проста процедура и чрез задаването на три точки позволява да се определи повърхността. Освен това възпроизвеждането на изображението на триъгълниците, тяхното запълване и определяне на двойките на пресичане не предизвикват затруднения. Основният недостатък на този метод е, че за да се създаде впечатление за гладкост на повърхността може да потрѣбват премного триъгълници. За крайни участъци могат да се използват линейни и билинейни интерполационни участъци (параметрични бикубични късове), повърхности на Безие и повърхности построени с В-сплайни.

11.2 ПРОЕКЦИИ НА ТРИМЕРНИ ОБЕКТИ ВЪРХУ ПЛОСКА ВИДОВА РАВНИНА

Генерираните координати на пикселите, представящи тримерните обекти в съответствие с методите за моделиране на последните, изложени в предния параграф следва да се изведат на съответното изходно графично устройство. Този процес на извеждане на тримерна графична информация е много по-сложен, отколкото съответния му двумерен процес, който бе обсъждан в предните лекции. По принцип *обектът се генерира в тримерни световни координати след което се отсича по границите на някакъв видим*

обем и едва след това следва да се преобразува така, че след като бъде изведен на екрана на дисплея да се получи илюзия за пространственост на този обект.

Сложността, която е характерна за тримерния случай се състои в това, графичната координатна система на изходното устройство няма трето измерение.

Несъответствието между пространствените обекти и плоските им изображения се отстранява с въвеждането на проекции, които и пренасят тримерните обекти на двумерната проекционна плоскост. Тук ще бъдат разглеждани само плоски геометрични проекции, защото се счита, че графичните изходни устройства са именно такива. Възможно е едно тяло да бъде проектирано и върху крива повърхност, което не е задача на настоящото разглеждане.

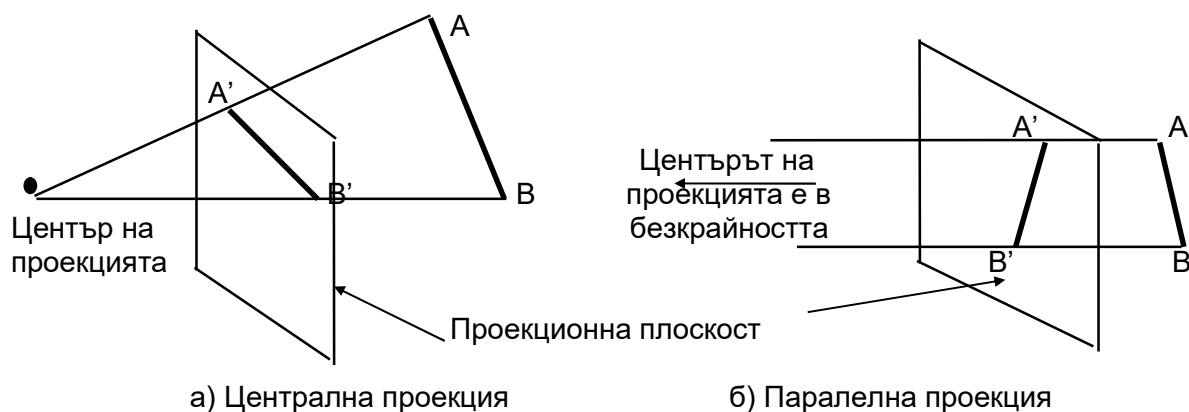
И така, с помощта на проектирането трите измерения на компютърно генерираните обекти следва да се представят чрез две координати.

Геометрични плоски проекциите се строят с помощта на **прави проекционни** лъчи, които се наричат проектори и които излизат от всяка точка от центъра на проекция, преминават през всяка точка на обекта и пресичайки картинната проекционна **плоскост**, образуват проекцията. Проекцията на отсечка се явява също отсечка и затова е достатъчно да се проектират само крайните точки.

Основните методи за получаване на геометрични, плоски проекции на тримерни обекти върху двумерната плоска равнина са два:

- Метод на паралелна проекция;
- Метод на централната или още перспективна проекция.

Основната разлика между двата метода е в съотношението между така



Фиг. 11.1

наречения център на проекцията и проекционната равнина. Ако разстоянието между тях е **безкрайност**, то имаме паралелна проекция, а ако е **крайно** това разстояние, то имаме централна проекция. На фиг. 11.1 е показан процеса на проектиране за двата случая. Съществуват и проекции върху не-плоски повърхнини и/или с проекционни лъчи, които не са прави а криви. За описанието на централната проекция се задава явно центъра на проекцията, а при паралелната само направлението на проектиране. Централната проекция поражда ефекта на перспективата и се използва за получаване на реалистични изображения. При нея размера на обекта се изменя обратно пропорционално на разстоянието му до центъра на проекция. Макар тази проекция да е реалистична, тя се оказва непригодна за представяне на точна форма и размер на обекта. В този случай от проекцията не може да се получи информация за разстоянията, ъглите на обекта. Съхраняват се само тези страни от обекта, които са паралелни на проекционната плоскост.. Проекциите на паралелни линии в общия случай не са паралелни. Паралелната проекция в общия случай не поражда реалистично изображение. Проекцията фиксира точните размери с точността на скаларния множител и се съхранява паралелността на линиите. Съхраняват се точно онези стени, които са паралелни на проекционната равнина.

11.2.1 Централна Проекция

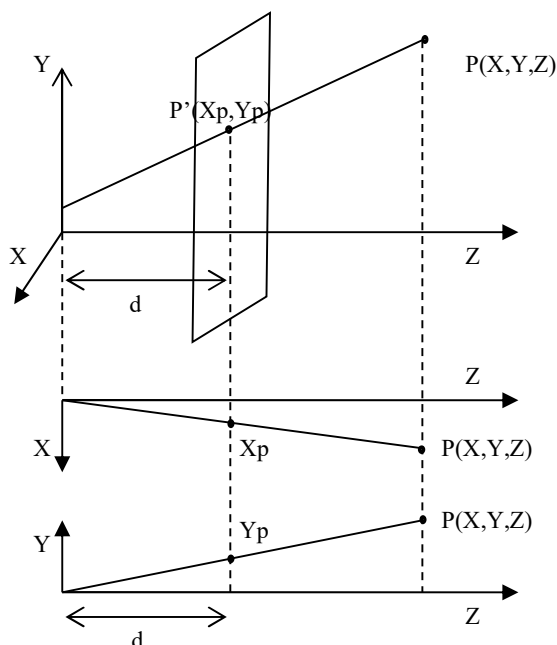
Съществуват едноточкова и дву-точкова централна проекция в зависимост от положението на проекционната плоскост. Говорим за едноточкова централна проекция, когато проекционната плоскост е перпендикулярна на оста Z и не пресича оста X. В случая точката

на сходимост е върху оста Z. Ако проекционната равнина пресича осите X и Z. Има две точки на сходимост по тези оси.

11.2.3 Математическо описание на плоските геометрични проекции

За простота ще считаме, че видовата равнина е перпендикулярна на оста z. При паралелната проекция също така тя съвпада с равнината $z=0$, а при централната проекция съвпада с равнината $z=d$.

А. Централна проекция



Фиг. 11.3. Централна проекция

Едноточкова централна проекция при която е прието, че координатите на центъра на проекцията са X_c, Y_c, Z_c и проекционна плоскост е перпендикулярна на Z където $Z=d$, то координатите на всяка точка $P(X, Y, Z)$ (Фиг.11.3.) от обекта оригинал ще се преобразуват в нови две координати X_p, Y_p на една двумерна точка, **представляваща, изобразяваща или пренасяща** точката оригинал върху проекционната равнина (11.5)

$$X_p = X_c - (Z_c - d) * \frac{X - X_c}{Z - Z_c} \quad Y_p = Y_c - (Z_c - d) * \frac{Y - Y_c}{Z - Z_c} \quad (11.5)$$

Тези две зависимости са получени като се използват елементарните съотношения страните на подобни триъгълници от геометрията на проблема.

В случай че плоскостта съвпада с равнината $Z=0$, и $(X_c, Y_c, Z_c) = (0, 0, -d)$ то са валидни съотношенията:

$$X_p = \frac{X}{Z/d + 1} \quad Y_p = \frac{Y}{Z/d + 1}$$

трансформационната матрица е $M_{\text{центр}} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1/d \\ 0 & 0 & 0 & 1 \end{bmatrix}$

11.2.2 Паралелна Проекция

Тези проекции са два типа в зависимост от това какво е съотношението между направлението на проектиране и нормалата към проекционната плоскост. В ортографичните паралелни проекции тези направления съвпадат, а при косо-ъгълните

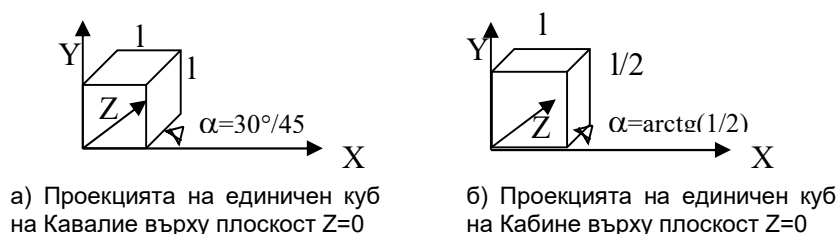
паралелни проекции не съвпадат. Най-използваните ортографични проекции са вид отпред, вид отзад, вид отгоре, вид отстрани, в които плоскостите на проектиране са перпендикулярни на главите координатни оси. При ортографичните проекции изображаването на някоя точка се определя като точка на пресичане на перпендикуляра спуснат от точката към проекционната плоскост. Ако тя е XY то Z се приравнява на нула или $Z=const$.

Случай на ортографична проекция има и когато проекционните плоскости не са перпендикулярни на координатните оси. В този случай говорим за аксиометрични проекции и при тях се изобразяват по няколко страни наведнаж. В този случай се съхранява паралелността на правите, но ъглите се изменят. Разстоянията могат да се измерват с точността на скалния коефициент.

Вид на аксио-метрична проекция е изо-метричната проекция. В този случай нормалата към проекционната плоскост и направлението на проекция съставляват равни ъгли с главните координатните оси. Изо-метричните проекции имат следните свойства. Трите оси се изменят еднакво и това позволява да се измерва еднакво и според използвания мащабен коефициент. В този случай осите се проектират като склучващи по 120° .

Косоъгълните проекции съчетават в себе си свойствата на ортографичните проекции със свойствата на аксиометрията. В този случай проекционната плоскост е перпендикулярна на главната координатна ос, затова страните на обекта, успоредни на тази плоскост се проектират така, че могат да се измерват ъгли и разстояния. Другите страни се проектират така, че да се провеждат линейни измервания. Две важни косо-ъгълни проекции са проекциите на Кавалие (Cavalier) и Кабине (Cabinet).

В проекцията на Кавалие (Фиг.11.2.а) направлението на проекцията съставя с плоскостта ъгъл 45° . В резултат на това проекцията на отсечка перпендикулярна на плоскостта има същата дължина каквато и отсечката, т.е. изменение отсъства. Тук линиите в дълбочина се явяват проекции на тези ребра на куба, които са перпендикулярни на плоскостта XY . Те са разположени под ъгъл α към хоризонталата. Този ъгъл обикновено е 30° или 45° .



Фиг. 11.2

Проекцията на Кабине, която има направление на проектиране, което склучва с проекционната плоскост ъгъл $\arctg(1/2)$ (Фиг.11.2.б). В този случай, отсечка, перпендикулярна на проекционната плоскост, след проектирането е $1/2$ от нейната действителна дължина. Тази проекция е по-реалистична, отколкото на Кавалие понеже изменение с $1/2$ по-добре се съгласува с нашия визуален опит.

Математическо представяне ортографична паралелна проекция

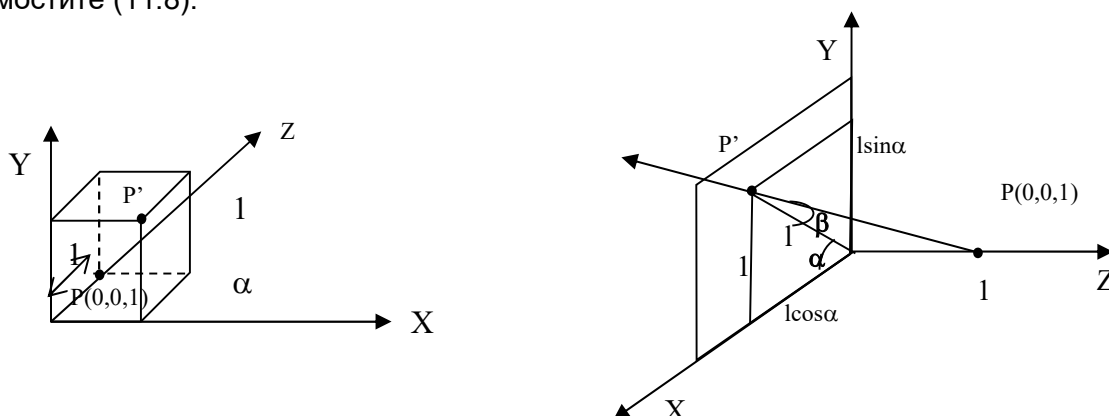
В частния случай на паралелна **ортографична проекция** върху равнина перпендикулярна на оста Z , координатите на всяка точка (x,y,z) от обекта оригинал се пренася **паралелно** върху видовата равнина, като очевидно стойностите на новите две координати x_p, y_p ще представляват старите такива, а координата z ще бъде равна на разстоянието на видовата равнина до равнината xOy , т.е. в случая това ще бъде величината d . Или:

$$X_p = X \quad Y_p = Y \quad Z_p = const = d \quad (11.6)$$

трансформационната матрица е $M_{ортг.} = \begin{vmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{vmatrix}$ (11.7)

В. Косо-ъгълна паралелна проекция

Единичният куб е проектиран на плоскост XY (Фиг.11.4). Проекцията на т. P(0,0,1), намираща се на зададената страна на единичния куб се явява т. P'(lcosα, lsinα, 0), принадлежаща на плоскостта XY. По определение това означава, че направлението на проектиране съвпада с отрязъка PP', преминаващ през тези две точки. P'-P=(lcosα, lsinα, -1) Ако направлението на проектиране сключва ъгъл β с плоскостта XY, то са валидни зависимостите (11.8):



Фиг. 11.4 Косоъгълна проекция

$$\frac{l}{1} = \cot g(\beta) \quad l = \frac{1}{\operatorname{tg}(\beta)} = \cot g(\beta) \quad (11.8)$$

$$X_p = X + Z(l \cos \alpha); \quad Y_p = Y + Z(l \sin \alpha)$$

където α (ъгълът между проекцията на лъча върху проекционната равнина и оста X;
l- дължината на проекцията. Матрицата на трансформация има вида:

$$M_{кос.} = \begin{vmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ l \cos \alpha & l \sin \alpha & 0 & 0 \\ 0 & 0 & 0 & 1 \end{vmatrix}, \text{ където } l = \cot g \beta = \frac{1}{\operatorname{tg} \beta} \quad (11.9)$$

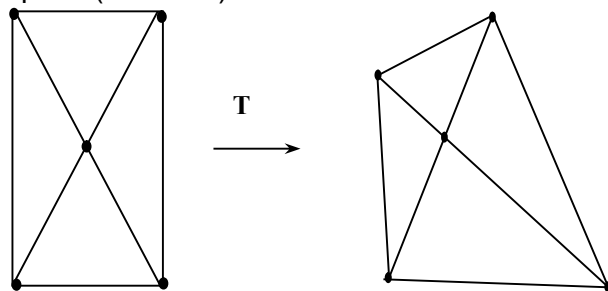
Прилагането на $M_{кос.}$ води до преместване и последващо проектиране на обекта по следния начин: плоскост с координати $Z=Z_1$ се пренася в направление X на $Z_1 l \cos \alpha$, а в направление Y на $Z_1 l \sin \alpha$ и последващо проектиране на $Z=0$. Преместването съхранява успоредността на правите, а така също и ъглите и разстоянията в плоскостите, успоредни на оста Z. За проекцията на Кавалие $l=1$, $\beta=45^\circ$. За проекцията на Кабине $l=1/2$, $\beta=\arctg(2)=63,4^\circ$. В случай на ортографична проекция $l=0$, $\beta=90^\circ$ и затова $M_{ортг.}$ е частен случай на $M_{кос.}$.

11.3. ПРОЕКЦИОННИ ПРЕОБРАЗОВАНИЯ

Проекционните преобразувания са общи линейни преобразувания T (представени в еднородни координати) (Фиг. 11.5.)

$$\begin{pmatrix} \bar{x}' \\ \bar{y}' \\ \bar{w}' \end{pmatrix} = \begin{pmatrix} t_{11} & t_{12} & t_{13} \\ t_{21} & t_{22} & t_{23} \\ t_{31} & t_{32} & t_{33} \end{pmatrix} \begin{pmatrix} \bar{x} \\ \bar{y} \\ \bar{w} \end{pmatrix}. \quad (11.10)$$

Проекционните преобразувания в общия случай не съхраняват паралелността на линиите. Свойството, което се съхранява при проектирането, е колинеарността, а именно 3 точки, лежащи на една права (т.е. са колинеарни), след преобразуването остават да лежат на една права (Фиг.11.5).



Фиг. 11.5. Действие на проекционно преобразуване на 5

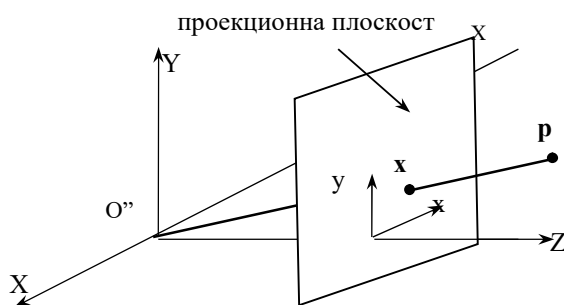
Затова проекционните преобразувания се наричат още *колинеация (хомография)*. От математическа гледна точка е удобно да се разглеждат реалния свят като включен в тримерното проекционно пространство P^3 , а плоскостта на изображението, като включена в проективното двумерно пространство P^2 . Точка от тримерната сцена и на изображението се представя в проекционното пространство като вектор в еднородни координати.

Проекционното преобразование от P^3 в P^2 , изобразяващо евклидовата точка от сцената $\mathbf{p}=(X,Y,Z)^t$ в точка от изображението $\mathbf{x}=(x,y)^t$, се задава с еднородни координати във вида:

$$\begin{pmatrix} \bar{x} \\ \bar{y} \\ \bar{w} \end{pmatrix} = \begin{pmatrix} wx \\ wy \\ w \end{pmatrix} = P \begin{pmatrix} WX \\ WY \\ WZ \\ Z \end{pmatrix} = \begin{pmatrix} p_{11} & p_{12} & p_{13} & p_{14} \\ p_{21} & p_{22} & p_{23} & p_{24} \\ p_{31} & p_{32} & p_{33} & p_{34} \\ p_{41} & p_{42} & p_{43} & p_{44} \end{pmatrix} \begin{pmatrix} \bar{X} \\ \bar{Y} \\ \bar{Z} \\ \bar{W} \end{pmatrix}. \quad (11.11)$$

Еднородните координати на вектора на проекционното пространство и проекционната плоскост се отнасят към нееднородните (евклидовите) вектори $\mathbf{p}$ и $\mathbf{x}$ по следния начин:

$$\mathbf{p} = \left\| \frac{\bar{X}}{\bar{W}}, \frac{\bar{Y}}{\bar{W}}, \frac{\bar{Z}}{\bar{W}} \right\|' \quad \text{и} \quad \mathbf{x} = \left\| \frac{\bar{x}}{\bar{w}}, \frac{\bar{y}}{\bar{w}} \right\|'.$$



Фиг. 11.6. Перспективна проекция

Проекционната геометрия е математическата основа на компютърната графика. Основните области на приложение са свързани с описанието както в процеса на формиране на изображението, така и при инвариантното представяне, а именно: калибровка на регистриращата камера, анализ на движение по серия изображения, разпознаване на образи, реконструкция на сцената по стереоснимки, синтез на изображения, анализ и

възстановяване на форми по полутонове. Композицията от две перспективни проекции не е задължително да бъде перспективна проекция, но определя проекционно преобразуване, т.е. проекционните преобразувания образуват група а переспективните проекции – не.

Изображението на снимка, формирано от регистрираща камера, е свързано с проектирането на сноп лъчи, доколкото всяка 2D точка е проекция на множеството 3D точки по продължение на лъча на проектиране в плоскостта на снимката (Фиг.11.6).

Нека развнината на снимката се описва с уравнението $Z=f$, където f е разстоянието между плоскостта и центъра на проекция. Тогава е валидна зависимостта между координатите на точка (x_i, y_i) от равнината и пространствените координати (X_i, Y_i, Z_i) :

$$x_i = \frac{fX_i}{Z_i}, \quad y_i = \frac{fY_i}{Z_i} \quad (11.12)$$

Лъчът, преминаващ през точката (x_i, y_i) , е направляващ вектор и се определя от правата $(0,0,0)$ и (x_i, y_i, f) . Върху нея лежи и тримерната точка $\lambda (x_i, y_i, f) = (\lambda x_i, \lambda y_i, \lambda f) = (X_i, Y_i, Z_i)$. Всъщност, всяка точка от изображението се определя от лъча, идващ от сцената в началото на координатите. Наборът от линии, преминаващ през началото на тримерното евклидово пространство E^3 , формира *проекционното* двумерно пространство P^2 в плоскостта $Z=f$.

В еднородни координати уравненията (11.12) имат вида:

$$\begin{pmatrix} wx_i \\ wy_i \\ w \end{pmatrix} = \begin{pmatrix} \overline{x_i} \\ \overline{y_i} \\ \overline{w} \end{pmatrix} = \begin{pmatrix} f & 0 & 0 & 0 \\ 0 & f & 0 & 0 \\ 0 & 0 & 1 & 0 \end{pmatrix} \begin{pmatrix} X_i \\ Y_i \\ Z_i \\ 1 \end{pmatrix} \quad .$$

11.4 OpenGL - 3D Трансформации

В **OpenGL** за тримерните обекти се използват трансформационни матрици 4x4. (за тримерни изображения в еднородни координати) **OpenGL** предоставя готови функции, с които може лесно да се работите с трансформационните матрици. В **OpenGL** съществуват три вида матрици: **PROJECTION**, **MODELVIEW** и **TEXTURE**. Те се използват съответно за определяне на изгледа в **3D** пространството, задаване на траекторията на обектите/камерата и модификация на дадена текстура. В даден момент се променя само една от матриците, по тази, която е включена преди това с функцията **glMatrixMode()**, приемаща един от аргументите: **GL_MODELVIEW**, **GL_PROJECTION** или **GL_TEXTURE** за съответната матрица.

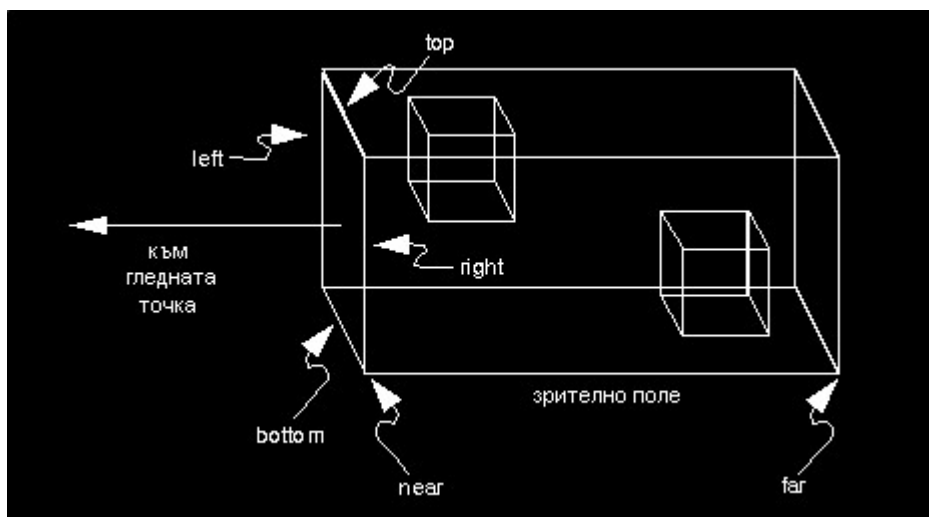
Използва се за дефиниране на изгледа (т.е. ако се изпусне тази стъпка във вашата програма ще виждате единствено черен екран, независимо от това, дали има нещо за изрисуване или не.) След включването на **PROJECTION** матрицата (например: **glMatrixMode(GL_PROJECTION);**), трябва да се дефинира вида на проекцията. Възможни за избор са два вида проекции, които се избират съответно с функциите :

void glOrtho(GLdouble left, GLdouble right, GLdouble bottom, GLdouble top, GLdouble zNear, GLdouble zFar) - ортографична перспектива (паралелна, с лъчи перпендикулярни на координатните оси)

void gluPerspective(GLdouble fovy, GLdouble aspect, GLdouble zNear, GLdouble zFar) - реалистична перспектива (централна проекция)

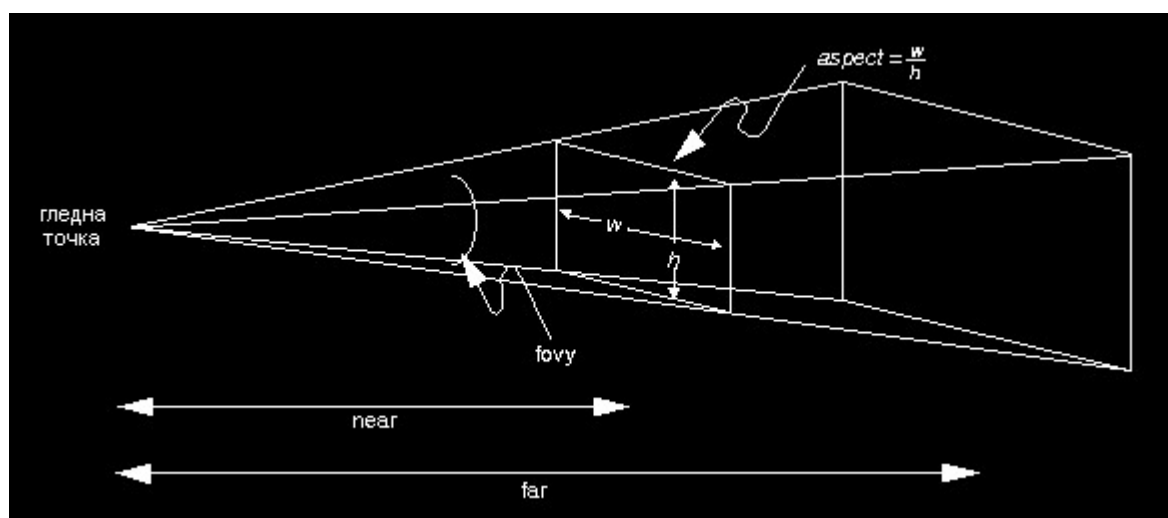
void glFrustum(GLdouble left, GLdouble right, GLdouble bottom, GLdouble top, GLdouble znear, GLdouble zfar) - реалистична перспектива(еквивалентна на горната функция).

Функцията **glOrtho()** изгражда т.н ортогографична перспектива, която съхранява дължините на страните и ъглите. Тя се отличава с това, че представя обектите по един и същи начин независимо от разстоянието им до камерата. Запазва паралелността на линиите и ъглите. Функцията приема 6 аргумента. Първите четири определят координатите на лява, дясна, долна и горна изрязващи равнини, а последните два аргумента определят съответно координатите на най-близкия и най-далечния обект, който ще бъде показан. Ако извикате **glOrtho** с координати **glOrtho(- 20 , 20 , - 20 , 20 , 1 , 100)** ще се построи един правоъгълник на перспективата на виждане със съответните координати. Всички обекти, които попадат в неговите параметри ще могат да бъдат виждани, а останалите - не. Всичко това може да видите ясно на следващата картинка.

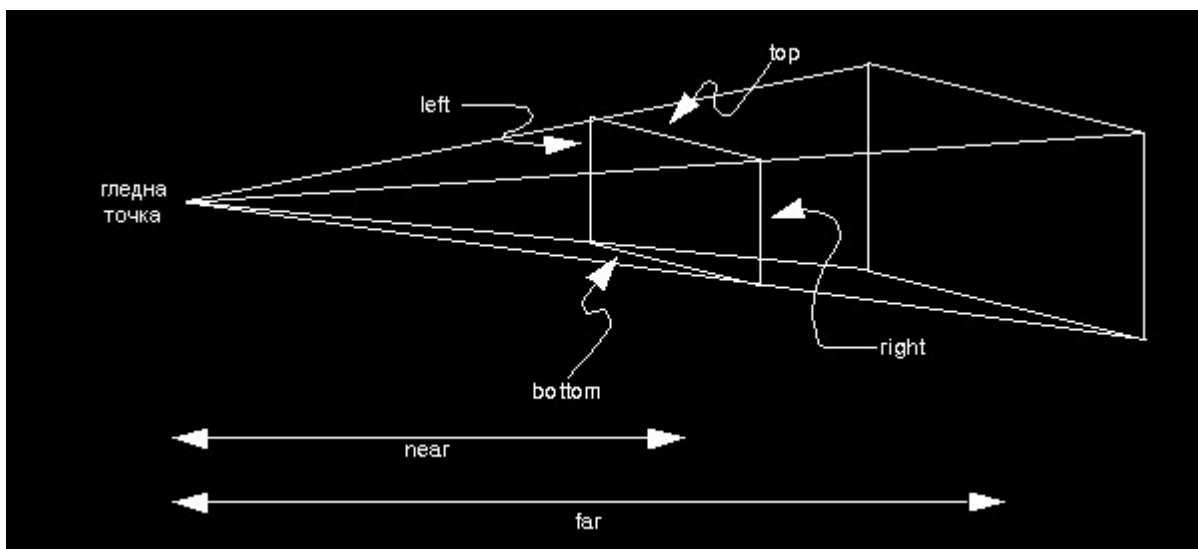


Другите две функции - **gluPerspective()** и **glFrustum()** създават реалистична перспектива, като генерират пирамида за перспективата, а не правоъгълник, т.е ако обекта е по-далеч, то той се вижда съответно по-малък и обратно. Двете функции представят централни проекция, но при различни входни параметри.

Функцията **gluPerspective()** приема четири аргумента. Първият определя ъгъла на виждане(fovy), който може да бъде от 0 - 180 (стандартно се избира 45 градуса), втория аргумент се генерира като се раздели ширината на проекционната плоскост на височина и (w/h). Най-често тази площ представлява квадрат и подадения аргумент е 1. Последните два аргумента са съответно най-близкото (near) и най-далечното разстояние (far) на което ще бъдат видяни различните обекти.



Функцията **glFrustum()** приема 6 аргумента. Първите четири аргумента определят размерите на проекционната плоскост при близко разстояние, а последните два аргумента - съответно най-близкото и най-далечното разстояние.



Дефинирането на изгледа трябва да се осъществи във функция, която се извиква веднъж след създаването на прозореца и преди започването на цикъла на съобщенията и на създаване на изображенията т.е във функцията **main()** ако се използва **GLUT** или при получаване на съответното съобщение от операционната система след създаването на прозореца, ако използвате **Win32 API**. Една добра перспектива е: **gluPerspective(45 , 1 , 1 , 200);**

В **OpenGL** има функция, която определя частта от прозореца, върху която ще се изрисова изображението :

void glViewport(GLint x, GLint y, GLsizei width, GLsizei height) - първите два аргумента определят координатите на долния ляв ъгъл, а останалите два аргумента - ширината и височината на пространството, в което ще се изобразява. Ако не се извика тази функция, то по подразбиране (**default**) прозореца съвпада със страницата. Тази функция е изключително важна при правилното оразмеряване на графиката в прозореца, при промяна на неговите размери след като вече е създаден.

MODELVIEW матрица:

Тази матрицата позволява да се преместват, въртят, мащабират различни обекти и да се определи позиция и посока на камерата. След като предварително е определен изгледа в **main()**, трябва да се превключи на **MODELVIEW** матрица преди да се започне непрекъснатото извикване на изобразяващата функция. Много е важно как се представя цялата сцена и нейните деформации.

трансформиране MODELVIEW матрицата:

glTranslate{fd}(x , y , z) - премества сцената/камерата в зависимост от подадените трансляционни координати.

glRotate{fd}(angle , x , y , z) - завърта сцената/камерата около (0,0,0) в равнините перпендикулярни на координатните оси на ъгъл **angle**. Например завъртане на обект на 30 градуса около x координатна ос трябва да извикате съответно: **glRotatetf (30 , 1 , 0 , 0);**

glScale{fd} (Sx , Sy , Sz) - мащабира сцената. Подадените аргументи се използват като коефициент на мащабирането по x , y или z координата. Стойност по-малка от 1 води до

смаляване на обекта, по-голяма от 1 - увеличаване на обекта. Ако някой аргумент е равен на 1, обектът запазва размерите си по съответната координата. Мащабирането е спрямо началото на КС (0,0,0).

Важно е да се отбележи, че всяка промяна на матрицата се отразява само на следващите изрисувания. Функцията **glLoadIdentity(void)** зарежда първоначалната матрица, по която не са правени никакви преобразувания. Например ако се завърти обект на 20 градуса и след това на 30 градуса, ще се получи обект завъртян на 50 градуса, ако не се извика **glLoadIdentity()** в началото на рендерираща функция. Всичко се дължи на това, че втората трансформация се прави върху вече трансформирана матрица. Ако това не е желателно, то тогава всеки път при извикване на рендериращата функция трябва да се зарежда и първоначалната матрица с **glLoadIdentity()**.

За да се определи позиция или посока на камерата трябва да се извика съответно **glTranslatef()** или **glRotatef()** в началото на изобразяващата функция, веднага след като се изчистят буферите и се зареди първоначалната матрица с **glLoadIdentity()**. За да се приближи камерата т.е да се придвижи напред трябва да се уголеми z координатата и обратно - ако се иска да се придвижи назад трябва да намалим z координатата. Стартовата позиция е (0 , 0 , 0). С **glRotatef()** може да се определи посоката на гледане като за да се погледне назад трябва да завъртим камерата на 180 градуса по x или y координата. Ако се цели да се отдалечи един обект от камерата (това е варианта с движението на цялата сцена при статична камера) трябва да се намали неговата z координата (аналогично все едно преместиме камерата назад) и обратно ако се цели приближаване нужно да се увеличи координатата. Ако се завърти сцената на 180 градуса всичко се обръща, т.е. като се намали z координатата, то обектите се приближават към статичната камера, а след това и зад нея.

Пример за преместване на камерата 10 единици назад (**glTranslatef (0 , 0 , -10);**), което може да се приеме и като преместване на сцената с обектите 10 единици напред.

void gluLookAt (eyex , eyey , eyez , centerx , centery , centerz , upx , upy , upz) - функцията приема 9 аргумента. Първите 3 определят местоположението на камерата (или преместването на цялата сцена около статичната точка на виждане), следващите 3 аргумента - координатите на гледната точка към която е насочена камерата(или завъртането на сцената), а последните 3 определят вектора, който показва кое е "нагоре". За нормална камера трябва да определите стойности за **upx** , **upy** и **upz** съответно 0 , 1 и 0.

В OpenGL има **3D** схващане на света и може да се разглеждат по два начина: с помощта на движеща се камера докато цялата карта (сцена) е статична или да се приеме варианта с движението на сцената с обектите около камерата.

Произволен брой отделни матрици **Modelview** могат да бъдат запазени в стек. Това се отнася също и за **PROJECTION** и **TEXTURE** матрицата. Важно е да се знае, че когато се слагат матрици в стека, те се поставят една върху друга и след това може да бъде извадена първо извадена само най-горната. Всякакви промени на матрица в стека се отразяват само на нея и на останалите матрици в стека след нея, ако има такива. Поставянето на матрица в стека става с функцията **glPushMatrix()**, а изваждането с **glPopMatrix()**. Когато се постави матрица в стека, после трябва да се извади за да се отразят съответните промени.

Примери :Ако трябва да се направят два куба, първият от които се върти по x и y координата, а втория - просто да стои до първия. Най-вероятно това ще представлява рисуващата функция :

1. **void RenderFunction()**
2. **{**
3. **glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);**
4. **glLoadIdentity();**
5. **angle++;**
6. **if (angle>=360) angle=0;**
7. **glTranslatef (0 , 0 , -50);** // определя преместване на камерата 50 единици назад
8. **glRotate (angle, 1 , 1 , 0);** // завърта сцената по x и y координата

```

9.   glutSolidCube( 2 ); // тази функция създава куб със страна 2 единици
10.  glTranslatef ( 2 , 0 , 0 ); // премества сцената две единици надясно
11.  glutSolidCube( 2 );
12.  glutSwapBuffers( );
13.  glutPostRedisplay( );
14.  }

```

Функцията **glRotatef()**, завърта цялата сцена. Всяка промяна на матрицата се отразява на бъдещите изрисувания, то тази промяна засяга и втория куб. Функцията **glTranslatef(2, 0, 0)** въздейства само на втория куб, понеже единствено той може да се приеме за бъдещо изрисуване. Ако се сложи завъртането и изрисуването на първия куб в стек, то ще се отрази само на него, без да обърка траекториите на останалите обекти, които ще се изрисуват след него. Например :

```

1.   void RenderFunction( )
2.   {
3.   glClear( GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT );
4.   glLoadIdentity( );
5.   angle++;
6.   if ( angle>=360 ) angle=0;
7.   glTranslatef ( 0 , 0 , -50 );
8.   glPushMatrix( ); // поставяме матрица в стека
9.   glRotate ( angle , 1 , 1 , 0 ); glutSolidCube( 2 );
10.  glPopMatrix( ); // изваждаме матрицата от стека
11.  glTranslatef ( 2 , 0 , 0 ); glutSolidCube( 2 );
12.  glutSwapBuffers( );
13.  glutPostRedisplay( );
14.  }

```

Ако трябва да се изобрази кола, която се състои от различни обекти (гуми, стъкла , врати и т.н.) и да се раздвижи, като едновременно с нея гумите се въртят и всичките части на колата се движат заедно с нея, то за тази цел трябва да се използват матрици, които се слагат една след друга в стека и после се вадят, като се започне от най-горната.

11.5 OPENGGL - КВАДРАТИЧНИ (ТРИМЕРНИ) ОБЕКТИ

Помощната библиотека **GLU** предоставя функции, с които могат да се създават някои по-сложни фигури като цилиндър, сфера, диск. Това са т.н. квадратични обекти. Също така могат да се зададат нивата на детайлност на всяка от фигурите като се зададат предварително броя на разделенията на обекта около оста z и по продължението на оста z. За да се построи квадратичен обект първо трябва да се създаде указател от тип **GLUquadricObj**, а след това да извикате функцията:

GLUquadricObj* gluNewQuadric(void) - тя връща указател към създадения квадратичен обект или нула, ако не е налична достатъчно оперативна памет. Например:

```

GLUquadricObj* object;
object=gluNewQuadric( );

```

След това е нужно да се определи режим на рисуване на обекта с функцията:

void gluQuadricDrawStyle(GLUquadricObj *quadObj, GLenum drawStyle) - първият аргумент трябва да е указателя сочещ създадения квадратичен обект (в случая това е **object**), а в втория аргумент може да бъде един от тези:

GLU_FILL - обектът се изобразява с множество многоъгълници

GLU_POINT - обектът се изобразява с множество точки (върховете на фигурата)

GLU_LINE - обектът се изобразява с множество отсечки

GLU_SILHOUETTE - обектът се изобразява като силует (подобно на режим с **GLU_LINE**)

Ако не се определи режим на рисуване, обектите се изобразяват, по подразбиране, с множество многоъгълници. Следва да се изобрази квадратичния обект, което се осъществява с една от следващите функции, от които зависи и каква фигура ще се визуализира:

void gluSphere(GLUquadricObj *quadObj, GLdouble radius, GLint slices, GLint stacks) - изобразява се сфера. Функцията приема 4 аргумента. Първият трябва да е указателя към създадения вече квадратичен обект, вторият аргумент - радиуса на сферата, третият и четвъртият - съответно броя на разделенията на обекта около и по продължението на оста Z.

void gluCylinder(GLUquadricObj * quadObj, GLdouble baseRadius, GLdouble topRadius, GLdouble height, GLint slices, GLint stacks) - изобразява се цилиндър. Функцията приема 6 аргумента. За първи аргумент се подава указател, асоцииран със свободен квадратичен обект, вторият аргумент - радиуса на основата, след това радиуса на горната основа и после височината на цилиндъра. Последните два аргумента са същите както по-горе.

void gluDisk(GLUquadricObj * quadObj, GLdouble innerRadius, GLdouble outerRadius, GLint slices, GLint loops) - изобразява се диск. Първият аргумент е идентичен, вторият и третият са съответно вътрешния и външния радиус, а последните два аргумента са броя на разделенията около оста z и броя на концентричните кръгове около центъра на диска.

void gluPartialDisk(GLUquadricObj * quadObj, GLdouble innerRadius, GLdouble outerRadius, GLint slices, GLint loops, GLdouble startAngle, GLdouble sweepAngle) - изобразява се частичен диск (сектор от диск). Функцията приема цели 7 аргумента. Първите пет аргумента, които трябва да се подадат са от предишните функции и единствено последните два са нови - **startAngle** и **sweepAngle** обозначават съответно началния ъгъл на изрисване и дължината на дъгата.

Ако се използва динамична светлина, ще се наложи да се определят **нормалите за всеки квадратичен обект**. Това става с функцията :

void gluQuadricNormals(GLUquadricObj *quadObj, GLenum normals) - първият аргумент представлява указателя към квадратичния обект, а за втори аргумент може да се подаде един от тези :

GLU_NONE - не се генерират нормали

GLU_FLAT - генерира се по една нормала за всяка стена

GLU_SMOOTH - генерира се по една нормала за всеки връх, което е и най-качествения режим

Също така можете да се определят накъде сочат нормалите, което може да бъде много полезно ако самата светлина се намира вътре в обект. Това става с функцията:

void gluQuadricOrientation(GLUquadricObj *quadObj, GLenum orientation) - първият аргумент е указател към обекта, а за втори може да се избере :

GLU_OUTSIDE - за външно насочени нормали

GLU_INSIDE - за вътрешно насочени нормали

Ако не се определи посока на нормалите по подразбиране ще бъдат създадени външно насочени нормали.

Когато даден квадратичен обект не е необходим то може да се освободите малко оперативна памет като се изтрие с функцията:

gluDeleteQuadric(GLUquadricObj *quadObj) - тя приема само един аргумент и това е указателя към квадратичния обект.